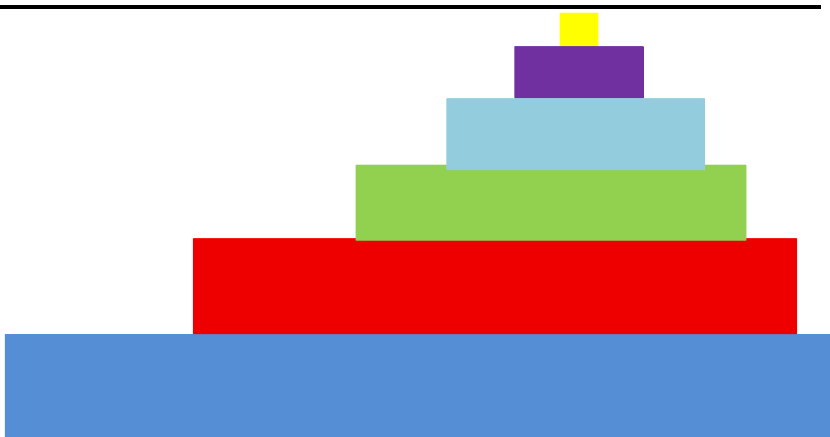




高级语言程序设计之

汉诺塔

大作业实验报告

实验报告名称：汉诺塔

班级：高程03班

学号：2452640

姓名：罗啦

完成日期：2026年5月27日星期三

1. 题目

1.1. 题目解读

本次作业要求将之前所有汉诺塔相关的小作业整合成一个完整的程序，实现以下功能：

1. 将递归解法、步数计算、字符图形输出等功能整合到同一程序中。
2. 通过菜单方式让用户选择不同的功能模块。
3. 在CMD窗口中实现简单的图形显示效果（字符/图形绘制）。
4. 实现盘子的平滑移动效果（上移→平移→下移）。

其中，各菜单的具体实现要求为：

菜单项	功能描述	呈现形式	关键要求
1	递归输出移动步骤	纯文字输出	只输出文字，无图形，与菜单 8 共用递归函数
2	带步数编号的输出	纯文字输出	只输出文字，无图形
3	横向输出（A/B/C 柱上的盘子编号）	文字表格形式	与菜单 4/8 共用横向输出函数
4	纵向输出 + 图形化演示	图文结合	与菜单 8 共用横向/纵向输出函数
5	画出三根圆柱	图形界面	需要加延时，与菜单 6-9 共用画圆柱函数
6	在指定圆柱上画出 n 个圆盘	图形界面	颜色各不相同，需要加延时
7	完成第一个圆盘的移动	图形动画	三步移动（上移-平移-下移）
8	完整汉诺塔自动演示	图形动画	调用递归函数，三步移动动画

菜单项	功能描述	呈现形式	关键要求
9	人工操作游戏	图形动画 + 交互	不调用递归，校验移动规则，三步移动动画

表1-1 菜单实现功能

1. 2. 基本要求概述

1. 2. 1. 项目结构要求

1. 项目命名：90-b1

2. 必须包含的九个文件：

文件名	作用	是否允许修改	是否需要提交
cmd_console_tools.cpp	控制台工具实现	否	否
cmd_console_tools.h	控制台工具声明	否	否
cmd_hdc_tools.cpp	图形工具实现	否	否
cmd_hdc_tools.h	图形工具声明	否	否
hanoi_const_value.h	常量定义	否	否
hanoi.h	项目头文件	是	是
hanoi_menu.cpp	菜单显示与选择	是	是
hanoi_multiple_solutions.cpp	各菜单项实现	是	是
hanoi_main.cpp	main 函数	是	是

表1-2 文件要求

1. 2. 2. 编程语言限制

1. 禁止使用的语法：指针，引用，结构体，类，goto语句，C++的string 类型，scanf/printf 输入输出函数等。

2.允许使用的全局变量:

用途	允许形式
总移动步数	1 个全局简单变量
圆柱上现有圆盘的编号	3 个全局一维数组 或 1 个全局二维数组
圆柱上现有圆盘的数量	3 个全局简单变量 或 1 个全局一维数组
延时时间	1 个全局简单变量

表1-3 允许使用的全局变量

1.2.3. 函数共用要求

序号	共用要求	涉及菜单项
1	一个递归函数（不超过 15 行）	1, 2, 3, 4, 8
2	一个输入函数	1, 2, 3, 4, 6, 7, 8
3	一个横向输出函数	3, 4, 8
4	一个纵向输出函数	4, 8
5	一个画圆柱函数	5, 6, 7, 8, 9
6	一个盘子移动函数	7, 8, 9

表1-4 函数共用要求

2. 整体设计思路

本次汉诺塔综合演示程序的设计遵循模块化与功能复用的原则，将整个项目划分为四个主要层次：用户交互层、业务逻辑层、功能实现层和工具支撑层。

用户交互层由hanoi_main.cpp负责，包含程序入口 main 函数、环境确认（检查控制台窗口大小与字体）以及主菜单循环。每次循环中，程序调用 hanoi_menu.cpp 中的菜单显示函数获取用户输入的选项，然后根据选项值调用 hanoi_multiple_solutions.cpp 中对应的执行函数，执行完毕后等待用户按回车键返回菜单，直到用户选择退出。

在功能实现层，所有菜单项的核心逻辑都放在 `hanoi_multiple_solutions.cpp` 中：

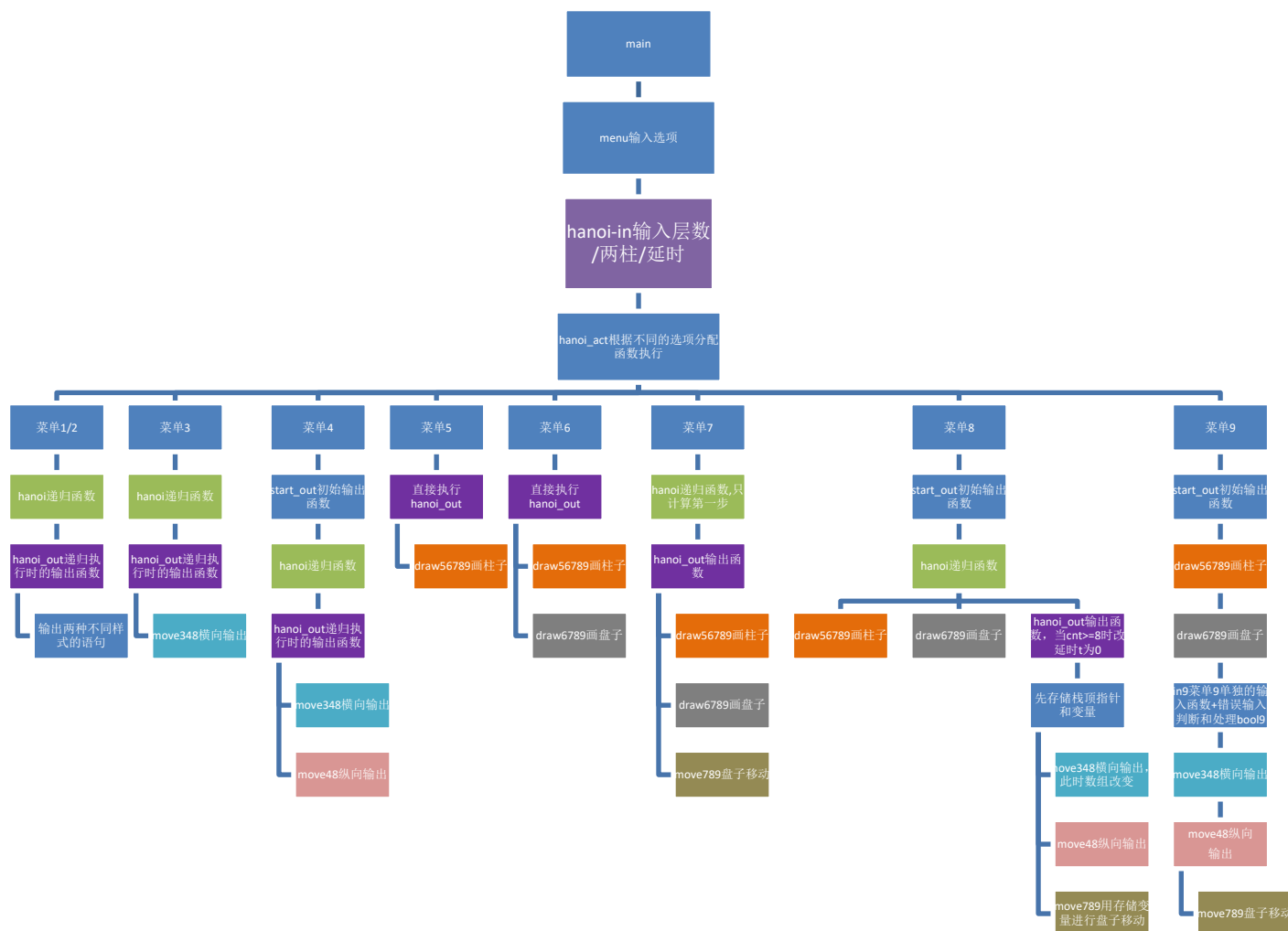
- (1) 一个递归函数 `hanoi`，它同时服务于菜单项1、2、3、4和8，通过一个参数来区分不同菜单的输出行为。该递归函数严格遵守不超过15行的限制，内部只负责调用输出调度函数，而不直接进行具体输出。
- (2) 一个统一的输入函数 `hanoi_in`，用于处理菜单项1、2、3、4、6、7、8中的参数输入（层数、起始柱、目标柱以及延时速度），该函数利用引用参数将多个输入值一次性带回，并包含了完整的错误处理机制（类型错误、越界输入等）。
- (3) 此外，横向输出函数 `move348` 被菜单项3、4、8共用，纵向输出函数 `move48` 被菜单项4和8共用，画圆柱函数 `drawPillars56789` 被菜单项5至9共用，盘子移动动画函数 `move789`被菜单项7、8、9共用。通过这种共用设计，有效避免了代码冗余，也使程序结构更加清晰。

数据结构方面，使用了作业允许的静态全局变量来记录游戏状态。具体包括一个一维数组 `topabc[4]`（索引1至3分别对应A、B、C柱）存储每根柱子上当前的圆盘数量，以及一个二维数组 `abc[4][11]` 存储每根柱子上从顶部到底部的圆盘编号（编号越小表示圆盘越小）。另外还设置了一个全局变量 `cnt`用于记录菜单项中的总移动步数，以及 `t`用于控制延时速度。

图形动画方面，程序利用题目提供的 `cmd_hdc_tools`库在控制台窗口中绘制矩形来代表圆柱和圆盘。所有绘制参数（颜色、坐标、宽度、高度、间距等）均从 `hanoi_const_value.h` 中读取，从而确保替换为胖版或瘦版常量文件后程序仍能正常显示。盘子的移动严格按照“上移—平移—下移”三步执行，每一步都采用“擦除原位置—在新位置绘制”的方式，通过循环逐步改变盘子的坐标，并在每一步之间调用 `Sleep`函数实现延时，从而形成平滑的动画效果。整个过程中严禁使用全屏清屏命令，以避免画面闪烁。

最后，对于菜单项9的人工操作游戏，该模块不调用递归函数，而是由用户输入两个字母（如 `AC` 表示从A柱移动到C柱），程序首先校验输入的合法性（源柱是否为空、目标柱顶部是否有更小的盘子等），然后调用共用的移动动画函数执行三步移动，并更新数据结构和步数计数器。当所有盘子都成功移动到目标柱时，程序提示游戏结束并返回主菜单。

3. 主要功能的实现



4. 调试过程碰到的问题

- (1) 因为之前的汉诺塔作业定位坐标是用生拼硬凑的坐标值代入的，因此移植到新代码时定位花费了很多时间。
- (2) 发现demo的菜单8是先横向输出纵向输出再开始移动圆盘的，而我的函数如果先做输出再移动的话会因为存放值的数组已经发生改变而导致移动错误。最后用两个局部变量分别存放输出之间栈顶指针和栈顶元素，再用这两个变量进行移动解决。
- (3) 为了理解画图工具的定位和使用花费了一点时间。
- (4) 处理输入错误花费了不少时间。

5. 心得体会

5.1. 完成本次作业得到的一些心得体会和经验教训

通过本次汉诺塔综合演示作业，我深刻体会到将多个小作业整合到一个大程序中所面临的挑战与收获。首先是代码整合的难度，之前每次小作业都是独立编写的，函数命名、变量风格都不统一，移植到新程序时需要大量修改和适配。这让我意识到从一开始就保持良好代码规范的重要性。其次是图形动画部分的实现，最初我完全不了解控制台绘图工具的用法，通过反复运行测试用例和阅读源码，才逐渐理解了坐标系统和绘制流程。这个过程中我学会了如何用擦除重绘配合延时循环来实现平滑移动，而不是简单粗暴地清屏重画。输入处理也是一个容易忽略但非常关键的环节，用户可能输入任何格式，程序必须具备足够的健壮性来应对错误输入并给出清晰提示。此外，函数共用要求让我重新思考了代码的组织方式，如何设计一个递归函数同时服务于多个菜单项，如何用参数区分不同行为，这些都是宝贵的经验。最后，时间管理上我也吸取了教训，前期花太多时间在细节优化上，导致后期调试比较紧张，以后应该先完成核心功能再逐步完善。

5.2 复杂程序的分小题与做大题的形式比较

在做一些较为复杂的程序时，我认为分为若干小题逐步完成比直接做大题更好。首先，分小题可以降低心理压力，每个小题的目标明确且规模可控，完成一个小题就能获得一次成就感，有助于保持学习动力。其次，分小题便于阶段性测试和调试，如果一口气写完所有代码再运行，面对上百行甚至上千行的代码很难快速定位错误。而每个小题独立完成并验证正确后，再整合到整体中，出错的概率和排查难度都大大降低。第三，分小题有助于理解问题本身，通过逐个解决子问题，对整体问题的理解也会更加深入。当然，分小题也有一个前提，就是在开始之前需要对最终目标有清晰的认识，否则各个小题可能各自为政，难以整合。本次作业就是很好的例子，从简单的递归输出到横向输出、纵向输出，再到图形绘制和动画，每一步都有明确的进阶关系，循序渐进地完成了复杂任务。

5.3 前后小题的关联关系与代码重用

在完成汉诺塔系列小题的过程中，我确实考虑了前后小题之间的关联关系。最开始做递归输出时，我并没有过多关注后续的图形化需求，但随着题目一步步深入，我逐渐意识到前面写的函数如果设计得过于具体，后面就很难复用。例如，最初我把输出移动步骤的代码直接写在递归函数里，但到了需要横向输出和纵向输出时，就必须修改递归函数的结构。总结经验，要实现更好的代码重用，应该注意以下几点：一是尽量将核心算法与输出方式分离，递归函数只负责计算移动顺序，具体的输出交给单独的函数处理；二是使用参数来区分不同的输出行为，而不是写多个功能重复的函数；三是在编写前面小题时，可以适当预留接口或使用条件判断，为后续扩展留出空间；四是及时重构，当发现某个函数需要被多个地方使用时，就应该把它提取出来放到共用位置。本次作业最终要求多个菜单项共用同一个递归函数，正是因为前期各小题之间缺乏关联设计，才需要通过大量修改来满足要求。

5.4 如何更好地利用函数来编写复杂程序

以本次作业为例，我认为要更好地利用函数来编写复杂程序，需要注意以下几个方面。第一，遵循单一职责原则，每个函数只做一件事。例如，输入函数只负责读取和验证输入，不参与业务逻辑；移动函数只负责动画执行，不关心移动是否合法。这样每个函数的逻辑都比较简单，易于理解和调试。第二，合理划分函数粒度，既不能把太多功能塞进一个函数导致代码冗长难读，也不能过度拆分导致函数之间调用关系过于复杂。题目建议每个函数不超过50行是一个很好的参考标准。第三，提取公共功能为共用函数，本次作业要求多个菜单项共用输入函数、横向输出函数、画圆柱函数等，这正是代码复用的体现。当一个功能被多处使用时，将其封装成独立函数可以大幅减少重复代码，也便于统一修改。第四，使用参数来增加函数的灵活性，同一个函数通过不同的参数可以表现出不同的行为。例如递归函数通过参数区分菜单类型，从而决定是输出纯文字还是输出图形。第五，函数命名要清晰直观，让人一眼就能看出函数的作用，避免使用含义模糊的名字。总之，函数是组织复杂程序的基本单元，合理运用函数可以让代码更加模块化、可读性更强、维护成本更低。

6. 附件：源程序

```

/*****
函数名称: hanoi
功    能: 汉诺塔核心函数
输入参数: 层数, 起始柱, 中间柱, 结束柱
返 回 值: void
说    明: 12348 共用, 不超过 15 行
*****/
void hanoi(int choose,int n, char src, char tmp, char dst)
{
    if (n == 1) {
        hanoi_out(choose,1, src, tmp, dst);
        if (choose == 7)
            return;
    }
    else {
        hanoi(choose,n - 1, src, dst, tmp);
        if (choose == 7)
            return;
        hanoi_out(choose, n, src, tmp, dst);
        hanoi(choose,n - 1, tmp, src, dst);
    }
}

/*****
函数名称: move48
功    能: 纵向移动 4/8
输入参数: 选择, 起始柱, 结束柱
返 回 值: void
说    明:
*****/

```

```
void move48(int choose, char src, char dst)
{
    int x, y;
    if(choose==4)
        x = MenuItem4_Start_X + Underpan_A_X_OFFSET, y = MenuItem4_Start_Y + Underpan_A_Y_OFFSET;
    else if(choose==8)
        x = MenuItem8_Start_X + Underpan_A_X_OFFSET, y = MenuItem8_Start_Y + Underpan_A_Y_OFFSET;
    else
        x = MenuItem9_Start_X + Underpan_A_X_OFFSET, y = MenuItem9_Start_Y + Underpan_A_Y_OFFSET;
    if (src == 65) {
        cct_gotoxy(x, y-2 - topabc[1]);
        cout << " ";
    }
    if (src == 66) {
        cct_gotoxy(x + Underpan_Distance, y-2 - topabc[2]);
        cout << " ";
    }
    if (src == 67) {
        cct_gotoxy(x + 2*Underpan_Distance, y-2 - topabc[3]);
        cout << " ";
    }
    if (dst == 65) {
        cct_gotoxy(x, y-1 - topabc[1]);
        cout << abc[1][topabc[1] - 1];
    }
    if (dst == 66) {
        cct_gotoxy(x + Underpan_Distance, y-1 - topabc[2]);
        cout << abc[2][topabc[2] - 1];
    }
    if (dst == 67) {
        cct_gotoxy(x + 2*Underpan_Distance, y-1 - topabc[3]);
        cout << abc[3][topabc[3] - 1];
    }
}
```

/*****

函数名称: move348

功 能: 横向移动 3/4/8

输入参数:

返 回 值: void

说 明:

*****/

```
void move348(int choose, int n, char src, char dst) {
    if (choose == 4) {
        cct_gotoxy(MenuItem4_Start_X, MenuItem4_Start_Y);
        if (t > 0)
            Sleep(t);
        else
        {
            while (1) {
                char h = _getch();
                if (h == 13)
                    break;
            }
        }
    }
    if (choose == 8) {
        cct_gotoxy(MenuItem8_Start_X, MenuItem8_Start_Y);
```

```

    if (t > 0 || (t == 0 && cnt >= 8))
        Sleep(t);
    else
    {
        while (1) {
            char h = _getch();
            if (h == 13)
                break;
        }
    }
}

if (choose == 9) {
    cct_gotoxy(MenuItem9_Start_X, MenuItem9_Start_Y);
    if (t > 0)
        Sleep(t);
    else
    {
        while (1) {
            char h = _getch();
            if (h == 13)
                break;
        }
    }
}

cout << "第" << setw(4) << cnt++ << " 步(" << setw(2) << n << " #: " << src << "—>" << dst << ") ";
int m;
switch (src) {
case 65:
    m = abc[1][topabc[1] - 1];
    abc[1][topabc[1] - 1] = 0;
    topabc[1]--;
    break;
case 66:
    m = abc[2][topabc[2] - 1];
    abc[2][topabc[2] - 1] = 0;
    topabc[2]--;
    break;
case 67:
    m = abc[3][topabc[3] - 1];
    abc[3][topabc[3] - 1] = 0;
    topabc[3]--;
    break;
}

switch (dst) {
case 65:
    abc[1][topabc[1]++] = m;
    break;
case 66:
    abc[2][topabc[2]++] = m;
    break;
case 67:
    abc[3][topabc[3]++] = m;
    break;
}

cout << " A:";
for (int i = 0; i < topabc[1]; i++) {
    cout << setw(2) << abc[1][i];
}

```

```

    }
    for (int i = topabc[1] + 1; i <= 10; i++) {
        cout << " ";
    }
    cout << " B:";
    for (int i = 0; i < topabc[2]; i++) {
        cout << setw(2) << abc[2][i];
    }
    for (int i = topabc[2] + 1; i <= 10; i++) {
        cout << " ";
    }
    cout << " C:";
    for (int i = 0; i < topabc[3]; i++) {
        cout << setw(2) << abc[3][i];
    }
    for (int i = topabc[3] + 1; i <= 10; i++) {
        cout << " ";
    }
    cout << endl;
}

```

函数名称: draw56789

功 能: 画柱子

输入参数:

返 回 值: void

说 明:

*****/

void draw56789()

```

{
    hdc_init(HDC_COLOR[0], 0, 1200, 1200);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(HDC_Start_X, HDC_Start_Y, bw, HDC_Base_High, HDC_COLOR[11]);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(HDC_Start_X+ bw +HDC_Underpan_Distance, HDC_Start_Y, bw, HDC_Base_High, HDC_COLOR[11]);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(HDC_Start_X + 2*bw+2*HDC_Underpan_Distance, HDC_Start_Y, bw, HDC_Base_High, HDC_COLOR[11]);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(HDC_Start_X+11*HDC_Base_Width, HDC_Start_Y-h, HDC_Base_Width,h, HDC_COLOR[11]);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(HDC_Start_X + bw + HDC_Underpan_Distance+ 11 * HDC_Base_Width, HDC_Start_Y-h,
HDC_Base_Width, h, HDC_COLOR[11]);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(HDC_Start_X + 2*bw + 2*HDC_Underpan_Distance + 11 * HDC_Base_Width, HDC_Start_Y - h,
HDC_Base_Width, h, HDC_COLOR[11]);
    Sleep(HDC_Init_Delay);
}

```

函数名称: draw6789

功 能: 画初始盘子

输入参数:

返 回 值: void

说 明:

*****/

void draw6789(char src) {

int sX = 0, sY;

```

int srcc = src - 'A' + 1;
sY = HDC_Start_Y;
switch (src) {
case 'A':
    sX = HDC_Start_X;

    break;
case 'B':
    sX = HDC_Start_X + bw + HDC_Underpan_Distance;
    break;
case 'C':
    sX = HDC_Start_X + 2 * bw + 2 * HDC_Underpan_Distance;
    break;
}
for (int i=0,j=1 ; i < topabc[srcc]; i++,j++) {
    hdc_rectangle(sX + (bw - (1 + 2 * abc[srcc][i]) * HDC_Base_Width) / 2, sY-j* HDC_Base_High, (1 + 2
* abc[srcc][i]) * HDC_Base_Width, HDC_Base_High, HDC_COLOR[abc[srcc][i]]);
    Sleep(HDC_Init_Delay);
}
}
}
/*****
函数名称: move789
功    能: 盘子移动
输入参数: 起始柱, 目标柱;
返 回 值: void
说    明:
*****/

void move789(char src, char dst,int tsrc,int srctop,int tdst) {
    int sX=0,dX=0;
    switch (src) {
    case 'A':
        sX = HDC_Start_X;
        break;
    case 'B':
        sX = HDC_Start_X + bw + HDC_Underpan_Distance;
        break;
    case 'C':
        sX = HDC_Start_X + 2 * bw + 2 * HDC_Underpan_Distance;
        break;
    }
    switch (dst) {
    case 'A':
        dX = HDC_Start_X;
        break;
    case 'B':
        dX = HDC_Start_X + bw + HDC_Underpan_Distance;
        break;
    case 'C':
        dX = HDC_Start_X + 2 * bw + 2 * HDC_Underpan_Distance;
        break;
    }
    int srcc = src - 'A' +1;
    int dstt = dst - 'A' +1;
    int locationX, locationY;//左上角标记
    int goalX, goalY;
    locationY = HDC_Start_Y - tsrc * HDC_Base_High;

```

```

locationX = sX + (23 - (1 + 2 * srctop)) / 2 *
HDC_Base_Width;
while (locationY >= HDC_Top_Y) { //向上
    hdc_rectangle(locationX, locationY+
HDC_Base_High- HDC_Step_Y, (1 + 2 * srctop) *
HDC_Base_Width, HDC_Step_Y, HDC_COLOR[0]);
    if(locationY>= HDC_Start_Y-h-
HDC_Base_High)
        hdc_rectangle(sX+11 * HDC_Base_Width,
locationY + HDC_Base_High -
HDC_Step_Y, HDC_Base_Width, HDC_Step_Y,
HDC_COLOR[11]);
    hdc_rectangle(locationX, locationY -
HDC_Step_Y, (1 + 2 * srctop) * HDC_Base_Width,
HDC_Step_Y, HDC_COLOR[srctop]);
    locationY -= HDC_Step_Y;
    if (t > 0 || (t==0&&cnt>=8))
        Sleep(t);
    else
    {
        while (1) {
            char h = _getch();
            if (h == 13)
                break;
        }
    }
}
goalX= dX + (bw - (1 + 2 * srctop) *
HDC_Base_Width) / 2;
goalY= HDC_Start_Y - (tdst+1) * HDC_Base_High;
if (goalX > locationX) { //向右
    while (goalX > locationX) {
        hdc_rectangle(locationX,
locationY, HDC_Step_X, HDC_Base_High,
HDC_COLOR[0]);
        hdc_rectangle(locationX+ (1 + 2 *
srctop) * HDC_Base_Width, locationY, HDC_Step_X,
HDC_Base_High, HDC_COLOR[srctop]);
        locationX += HDC_Step_X;
        if (t > 0 || (t == 0 && cnt >= 8))
            Sleep(t);
        else
        {
            while (1) {
                char h = _getch();
                if (h == 13)
                    break;
            }
        }
    }
}
else {
    while (goalX < locationX) { //向左
        hdc_rectangle(locationX- HDC_Step_X,
locationY, HDC_Step_X, HDC_Base_High,
HDC_COLOR[srctop]);

```

```

        hdc_rectangle(locationX + (1 + 2 *
srctop) * HDC_Base_Width- HDC_Step_X, locationY,
HDC_Step_X, HDC_Base_High, HDC_COLOR[0]);
        locationX -= HDC_Step_X;
        if (t > 0 || (t == 0 && cnt >= 8))
            Sleep(t);
        else
        {
            while (1) {
                char h = _getch();
                if (h == 13)
                    break;
            }
        }
    }
}
while (locationY < goalY) { //向下
    hdc_rectangle(locationX, locationY, (1 + 2
* srctop) * HDC_Base_Width, HDC_Step_Y,
HDC_COLOR[0]);
    if (locationY >= HDC_Start_Y - h )
        hdc_rectangle(dX + 11 * HDC_Base_Width,
locationY, HDC_Base_Width, HDC_Step_Y,
HDC_COLOR[11]);
    hdc_rectangle(locationX, locationY +
HDC_Base_High, (1 + 2 * srctop) * HDC_Base_Width,
HDC_Step_Y, HDC_COLOR[srctop]);
    locationY += HDC_Step_Y;
    if (t > 0 || (t == 0 && cnt >= 8))
        Sleep(t);
    else
    {
        while (1) {
            char h = _getch();
            if (h == 13)
                break;
        }
    }
}
}
}

```